to value stored in a large database that is stored on a disk is a very time consuming process.

searching an un-indexed and unsorted database containing n key values needs O(n) running time.

6). B + Tree :-

B+tree is an extension of B tree which allows efficient insertion, deletion and search operations.

The leaf nodes of B+ tree are linked together in form of the singly linked list to make search queries more efficient.

Advantages of B+ tree :-

1). Records can be fetched in equal number of disk accesses.

2). Height of tree remains balanced and less as compare to B tree.

3). We can access data stored in B+ tree sequentially as well as directly.

4). Keys are used for indexing.

Graph :-

A graph can be defined as group of vertices and edges that are used to connect these vertices.

Defination :-

A graph G can be defined as an ordered set G(V, E) where V(G) represents set of vertices and E(G) represents set of edges.

which are used to connect these vertices.

Directed and Undirected Graph :-

A graph can be directed or undirected. However, in an undirected graph, edges are not associated with directions with them.

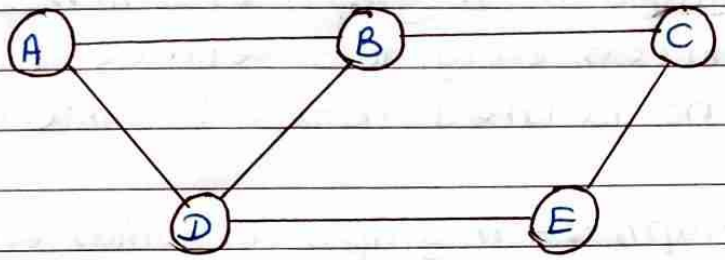


fig : Undirected graph

As above figure edges are not attached with any of the directions.

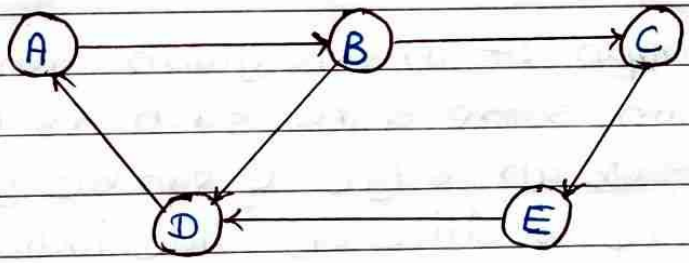


fig: directed graph.

In above figure, directed graph edges form an ordered pair.

Graph Terminology :-

1). Path :- A path can be defined as sequence of nodes that are followed in order to reach some terminal node v from initial node v.

2). closed path :- A path will be called as closed if initial node is same as terminal node. $V_0 = V_N$

3). Simple path :- If all nodes of graph are distinct with an exception $V_0 = V_N$, then such path p is called as closed simple path.

4). Cycle :- A cycle is a path which has no repeated edges or vertices except first and last vertices.

5). Connected graph :- A graph in which some path exists between every two vertices (u, v) in V. There are no isolated nodes in connected graph.

6). Complete graph :- A graph in which every node is connected with all other nodes. A complete graph contain $\frac{n(n-1)}{2}$ edges where n is number of nodes in graph.

7). Weighted graph :- In this graph each node is assigned with some data such as length or width The weight of an edge e can be given as w(e) which must be positive (+) value indicating cost of traversing edge.

8). Diagraph :- A diagraph is directed graph in which each edge is associated with some direction and traversing can be done only in specified direction.

9). Loop :- An edge that is associated with the similar end points can be called as loop.

10). Adjacent Nodes :- If two nodes u and v are connected via an edge e, then nodes u and v

are called as neighbours or adjacent nodes.

11). Degree of a Node :- A degree of a node is a number of edges that are connected with that node. A node with degree 0 is called isolated.

Graph Representation :-

We simply mean, technique which is to be used to in order to store some graph into the computers memory.

(1). Sequential Representation :- In this we use adjacency matrix to store mapping represented by vertices and edges. A graph having n vertices, will have a dimension n x n.

An entry Mij in adjacency matrix representation of an ~~not~~ undirected graph G will be 1 if there exists an edge between Vi and Vj.

An undirected graph and its adjacency matrix representation is shown in following :

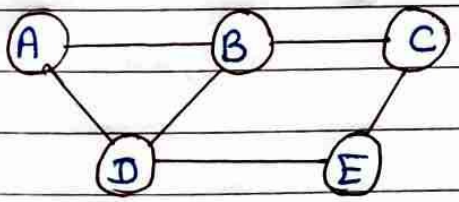


fig : Undirected graph

	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	1	0
C	0	1	0	0	1
D	1	1	0	0	1
E	0	0	1	1	0

Fig: Adjacency matrix

In above figure, we can see mapping among vertices (A, B, C, D, E) is represented by using adjacency matrix which is also shown in fig.

A directed graph and its adjacency matrix representation is shown in figure :

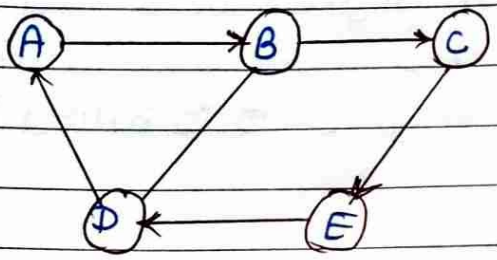


Fig : Directed Graph

	A	B	C	D	E
A	0	1	0	0	0
B	0	0	1	1	0
C	0	0	0	0	1
D	1	0	0	0	0
E	0	0	0	1	0

fig: Adjacency matrix

Representation of weighted directed graph is different. Instead of filling entry by 1, non zero entries of adjacency matrix are represented by weight of respective edges.

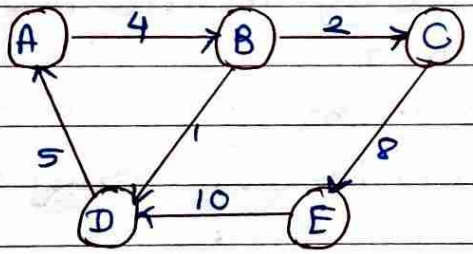


Fig : weighted directed graph

	A	B	C	D	E
A	0	4	0	0	0
B	0	0	2	1	0
C	0	0	0	0	8
D	5	0	0	0	0
E	0	0	0	10	0

fig: Adjancy matrix

(2) linked representation :-

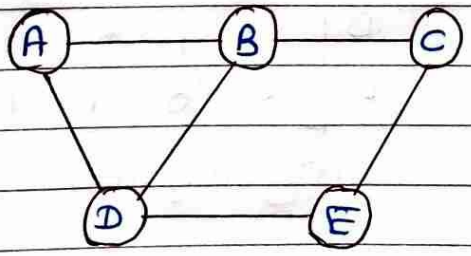


fig : undirected graph

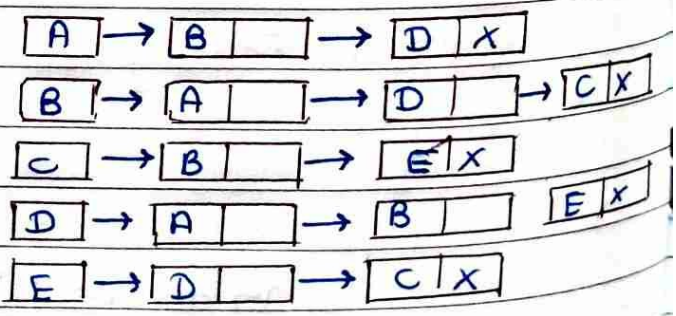


fig : Adjacency list.

An adjacency list is maintained for each node present in graph which stores node value and a pointer to next adjacent node to respective node.

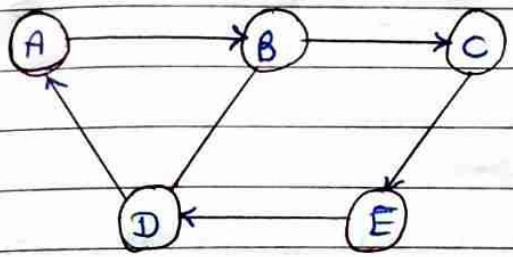


fig : Directed graph

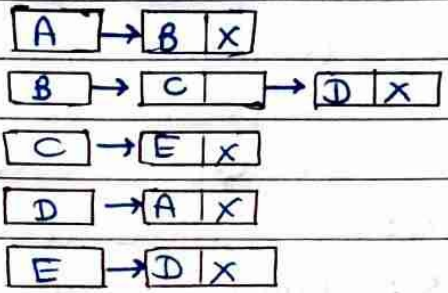


fig : Adjacy list

In directed graph, sum of lengths of all the adjacy lists is equal to the number of edges preseht in the graph.

Graph Traversal Algorithm :-

In this tutorial we will learn all techniques by using which, we can traverse all the vertices of the graph. Traversing means examining all nodes and vertices of graph. There are two standard methods by using which, we can traverse graphs.

- Breadth first search
- Depth first search

1. Breadth first search (BFS) algorithm :-

Breadth first search is a graph traversal algorithm that starts traversing graph from root node and explores all the neighbouring nodes.

Then, it selects nearest node and explore all unexplored nodes. The algorithm follows same process for each of nearest node until it finds goal.

Algorithm :-

step 1: SET STATUS = 1 (ready state) for each node in G.

step 2: Enqueue starting node A & set its STATUS = 2 (waiting state).

step 3: Repeat steps 4 and 5 until QUE E is empty.

step 4: Dequeue a node N. Process it & set its STATUS = 3

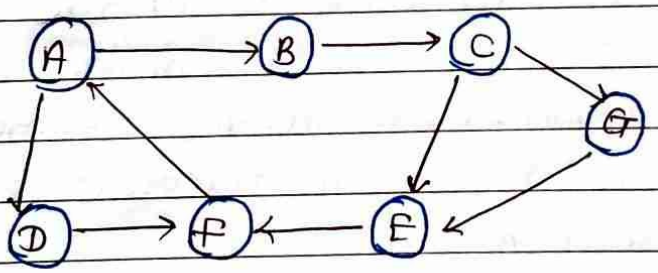
step 5: Enqueue all neighbours of N that are in ready state (whose STATUS = 1) & set (STATUS = 2) [END OF LOOP].

step 6: EXIT.

Example :-

Consider graph G shown in following image, calculate minimum path P from node A to node E. Given that each edge has a length of 1.

→



Adjacency lists :

A : B, D
B : C, F
C : E, G
G : E
E : B, F
F : A
D : F.

Solution :-

Minimum Path P can be found by applying Breadth first search algorithm that will begin at node A and will end at E.

A → B → C → E

Depth first search Algorithm :-

DFS algorithm starts with initial node of graph G, & goes to deeper & deeper until we find goal node / node which has no children. The data structure used in DFS is stack.

Algorithm :-

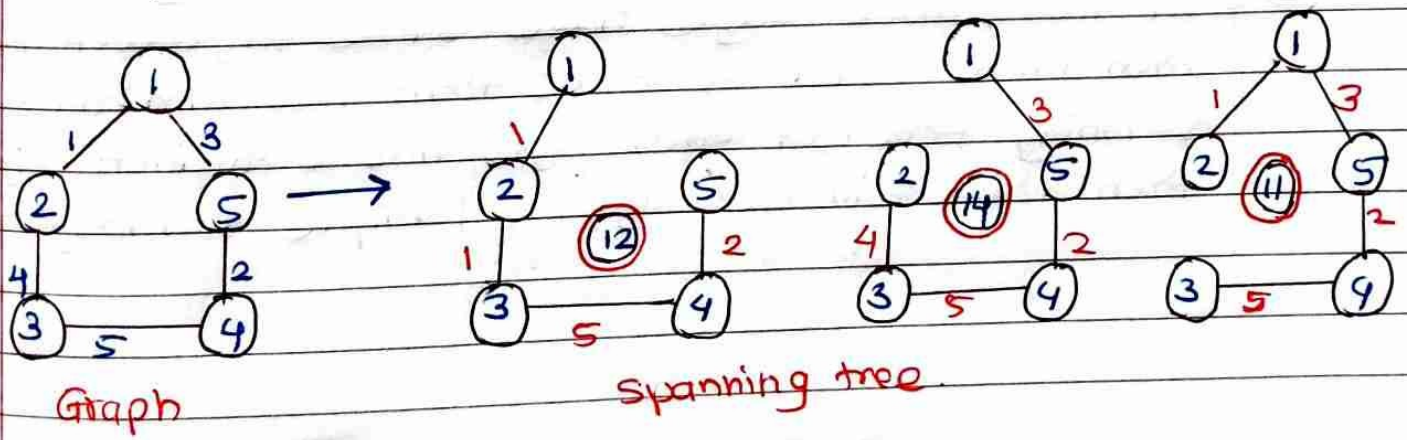
step 1: SET STATUS = 1 (ready state) for each node in G.
step 2: Push starting node A on stack & set its STATUS = 2 (waiting state).
step 3: Repeat steps 4 and 5 until stack is empty.
step 4: Pop top node N. Process it & set its STATUS = 3.
step 5: Push on stack all neighbours of N that are in ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state) [END OF LOOP].
step 6: EXIT.

Spanning Tree :-

If we have a graph containing V vertices and E edges, then graph can be represented as: G(V, E). If we create spanning tree from above graph, then spanning tree would have same number of vertices as the graph, but vertices are not equal.

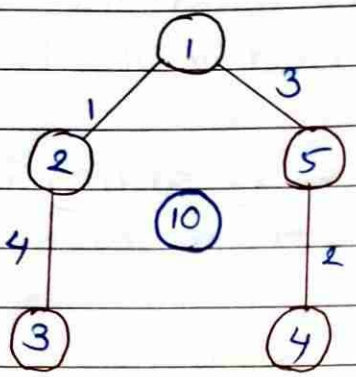
edges (spanning tree) = no. of edge (in graph) - 1.

Example :-



Minimum Spanning Trees :-

The minimum spanning tree is a tree whose sum of edge weights is minimum.



In above tree, total edge weight is less than above spanning trees, therefore a minimum spanning tree is a tree which is having an edge weight as 10.

Properties of Spanning tree :-

- A connected graph can contain more than one spanning tree.
- All possible spanning trees that can be created from given graph G would have same number of vertices in given graph minus 1.
- Spanning tree does not contain any cycle, let's understand this property through an example.
- Spanning tree cannot be disconnected. If we remove one more edge from any of above spanning trees as.
- If two / more edges have same edge weight, then there would be more than two minimum spanning tree. If each edge has a distinct weight, then there will be only one / unique spanning tree.

Applications of Spanning tree :-

1). Building a network :- Suppose there are many routers in network connected to each other, so there might be a possibility that it forms a loop.

2). Clustering :- clustering means that grouping set of objects in such way that similar objects belong to same group than to different group. our goal is to divide the n objects into k groups such that distance between different groups gets maximised.

Searching :-

Searching is a process of finding some particular element in list. If the element is present in the list, then process is called successful and process returns location of that element, otherwise search is called unsuccessful.

There are two methods widely used as below:

- Linear search
- Binary search

1). Linear search :-

linear search is a simplest sequential search algorithm and often called sequential search. In this type of searching, we simply traverse the list completely and match each element of list with item whose location is to be found.

linear search is mostly used to search an unordered list in which items are not sorted. The algorithm is given as follows :